



Tenable Vulnerability Management Connector — Credential Provisioning Guide

Version 0.1.0, 2026-09-09

Table of Contents

Overview.....	1
What kind of account to create.....	1
Permissions to grant	1
Generating the credential.....	2
Mapping values into Crogl.....	2
Verifying the setup.....	3
Reference documentation	3

Overview

The draft Crogl Tenable Vulnerability Management (Tenable.io) connector requires one API key pair — an access key and a secret key — generated from a Tenable Vulnerability Management user account. This guide is based on Tenable's published developer documentation and has not been verified against a live customer console. A Tenable Vulnerability Management administrator, or a user whose account already has at least the Basic user role, should perform the vendor-side steps.

IMPORTANT

This connector is **draft**: no live Tenable.io tenant has exercised it yet, and this guide's console steps (menu labels, exact click path) are inferred from Tenable's own written documentation rather than a first-hand walkthrough of the UI. Confirm the steps below against the customer's actual My Account page before relying on them, and do not mark the connector **field-tested** until a live query has returned real data.

NOTE

This is specifically for **Tenable Vulnerability Management** (cloud.tenable.com), not Tenable Security Center (a separate, typically self-hosted product with its own API and its own credential shape) or Tenable Identity Exposure (formerly Tenable.ad). If the customer's Tenable deployment is one of those instead, this guide and the paired connector do not apply.

What kind of account to create

Tenable Vulnerability Management does not have a separate "service account" or "app registration" concept for API access — an API key pair is generated directly from an existing user account's own **My Account** settings, and inherits that user's permissions and user role. There is no admin-approval step to create the key pair itself, but the account it belongs to still needs the right user role (see Permissions below).

Tenable recommends generating the key pair from an account dedicated to the integration (rather than a named individual's personal login) so the connector's access doesn't depend on — or get revoked alongside — one person's account. Whether a dedicated integration user is practical depends on the customer's own Tenable licensing and admin practices; if not, a named admin's account works, with the understanding noted below that regenerating that user's key later will break this connector.

Permissions to grant

Tenable Vulnerability Management uses container-wide user roles rather than a per-integration scope picker. The `GET /workbenches/vulnerabilities` and `GET /workbenches/vulnerabilities/{id}/outputs` endpoints this connector calls require the account to hold at least the **Basic** user role (Tenable's numeric role level 16) within the Tenable Vulnerability Management container. There is no separate consent step beyond the account itself having that role — a Tenable administrator grants or confirms the role under user management in the Tenable console.

NOTE

Tenable Vulnerability Management's role model is not divided into "Delegated vs. Application"-style permission types the way Microsoft Graph is — there is one user role per account, and the API key pair simply carries whatever that role already grants. There is no separate permission picker to misconfigure at key-creation time.

Generating the credential

1. Sign in to cloud.tenable.com as the account that will own the connector's credential (see What kind of account to create above).
2. Select the user-profile icon in the upper-right corner and open **My Account**.
3. Select **API Keys** from the left-hand navigation.
4. Select **Generate**, and confirm the action when prompted.
5. Copy both the displayed **Access Key** and **Secret Key** values immediately into an approved secret store. Tenable displays the secret key only once, at generation time — after leaving the My Account page it cannot be viewed again (only regenerated).

IMPORTANT

Each Tenable Vulnerability Management account can hold only **one** active API key pair at a time. Generating a new pair immediately invalidates the previous one. If this account's key pair is regenerated later for any reason (rotation, a different integration reusing the same account), this connector's stored credential breaks silently until it's updated with the new pair — prefer a dedicated integration account precisely to avoid this collision with a human's own key usage.

Mapping values into Crogl

Crogl field	Where it came from	Notes
access_key	The Access Key value shown when the API key pair was generated	Not treated as a Crogl secret field (see the connector's SKILL.md "Auth recap" for why — Tenable requires both values combined in one HTTP header, and only one of the two can be stored as an encrypted secret). Still don't paste it anywhere outside the Crogl connector setup form.
secret_key	The Secret Key value shown at the same time — displayed once	Treat as a secret. Cannot be recovered from Tenable after the generation screen is closed; only regenerated (which also invalidates access_key , requiring both fields to be re-entered together).

Verifying the setup

After creating the connector instance in Crogl, run a small read-only query, e.g. `tenable_io.py query --filter severity eq Critical --limit 5`, or use Crogl's built-in connector test-connection check (the connector declares a probe against `GET /api/v3/access-control/permissions/users/me`).

- **HTTP 200** — the key pair authenticated and the account has at least read access; an empty vulnerability result for a narrow filter is still a valid successful query, not a failure.
- **HTTP 401** — the `X-ApiKeys` header was missing, malformed, or the key pair is invalid/has been invalidated by a later regeneration. Re-generate the pair (see Generating the credential above) and update both `access_key` and `secret_key` in Crogl together — a stale `access_key` paired with a fresh `secret_key` (or vice versa) will also 401.
- **HTTP 403** — the key pair authenticated, but the owning account's Tenable user role is below Basic [16]. Have a Tenable administrator raise the account's role rather than re-entering the same credential.
- **HTTP 429** — Tenable is rate-limiting the account (or a concurrency limit was hit); this is not a credential problem. Wait for the interval named in the response's `Retry-After` header before retrying.

Do not use the example key values shown in Tenable's own API documentation pages — they are illustrative only and are not valid credentials.

Reference documentation

- Generate an API Key — <https://developer.tenable.com/docs/api-access>
- Authorization (`X-ApiKeys` header format) — <https://developer.tenable.com/docs/authorization>
- Generate API Keys (console walkthrough) — <https://docs.tenable.com/vulnerability-management/Content/Settings/my-account/GenerateAPIKey.htm>
- List vulnerabilities (`GET /workbenches/vulnerabilities`, required role) — <https://developer.tenable.com/reference/workbenches-vulnerabilities>
- List plugin outputs (`GET /workbenches/vulnerabilities/{id}/outputs`) — <https://developer.tenable.com/reference/workbenches-vulnerability-output>
- Workbench Filters — <https://developer.tenable.com/docs/workbench-filters>
- Rate Limiting — <https://developer.tenable.com/docs/rate-limiting>
- CVSS vs. VPR score ranges — <https://docs.tenable.com/vulnerability-management/Content/Explore/Findings/RiskMetrics.htm>
- Get current user permissions (probe endpoint) — <https://developer.tenable.com/reference/io-v3-access-control-permissions-current-user-list>